A Reproducible Data Workflow for the Iris Dataset

Ewald Geschwinde
AI Programming Foundations, Capstone Module Summary
17 October 2026

Overview

This project implements a complete data workflow in a single Jupyter notebook, taking the Iris dataset from a raw CSV file through cleaning, exploratory analysis, and visualization to a written interpretation. The dataset is Iris from the UCI Machine Learning Repository (https://archive.ics.uci.edu/dataset/53/iris), the 150-flower measurement table first published by Fisher (1936). The guiding question is how well four morphological measurements distinguish three iris species. The short answer is that they do so very unevenly: petal measurements separate the species almost completely, while sepal measurements barely separate them at all.

Dataset Description

The dataset records four measurements, all in centimetres, for 150 iris flowers: sepal length, sepal width, petal length, and petal width. The sepal is the outer, leaf-like structure that encloses the flower bud; the petal is the inner, usually more colourful structure. Each flower also carries a species label, giving five columns in total. The three species, Iris setosa, Iris versicolor, and Iris virginica, are represented by exactly 50 flowers each, so the classes are perfectly balanced by design rather than by chance.

All four measurements were treated as the variables of interest, since the project's question is precisely which of them carry discriminating information. The species label serves as the grouping variable throughout. The dataset is small and complete, with no missing values anywhere, which means the cleaning effort described below went into correctness rather than into imputation.

Workflow Description

The notebook is organised as five sequential stages, each in its own section, so that a reader can follow the data from file to conclusion without jumping around. Rule et al. (2019) recommend exactly this structure for shared computational analyses: short, single-purpose cells interleaved with explanatory narrative, so that the document explains itself to a reader who was not present when it was written.

- Ingestion. A single function reads `iris.csv` from the project folder and returns the data unmodified. The file is the UCI distribution file with a header row added and no values altered, and a separate utility, described below, can re-download the published file and verify the local copy against it on demand. Parsing is kept deliberately separate from cleaning so that the untouched input stays available for comparison.
- Cleaning. Three documented functions run in sequence: one normalises the species labels, one applies the transcription corrections published by the dataset authors, and one validates the measurements and reports data-quality problems. Each returns a new DataFrame rather than modifying its input, so every intermediate state can be inspected.
- Exploratory analysis. Four functions produce, respectively, an overall distribution summary, a per-species comparison of means and standard deviations, a correlation matrix, and a standardised measure of how cleanly each measurement separates each pair of species.
- Visualizations. Four figures, each built to answer one specific question, each carrying a title and labelled axes with units.
- Summary. A written interpretation of the findings, the limitations of the sample, and the questions that remain open.

Repeated logic was factored into named functions with docstrings rather than copied between cells, which is the second of the ten rules given by Rule et al. (2019) and the reason the cleaning steps can be re-applied or tested independently. Organising the analysis as a notebook also means the question, the code, the output, and the narrative live in one document, which is the literate-programming model that Danchev (2022) identifies as the central pedagogical advantage of Jupyter for teaching reproducible data science.

Key Decisions and Assumptions

Cleaning: correcting rather than deleting

Iris is often treated as a dataset with nothing to clean, and it is true that it has no missing values. Auditing it properly nevertheless turned up three problems, and the most consequential decision in this project was recognising that they called for three different responses.

The first problem was cosmetic: every species label carried a redundant `Iris-` prefix. The cleaning function strips it, lowercases the result, and converts the column to an ordered categorical so that every table and figure lists the species in the same sequence.

The second problem was substantive. The documentation shipped with the dataset, the file `iris.names`, records that samples 35 and 38 were transcribed incorrectly from Fisher's original paper, and states the correct values. In the distributed file both rows read 4.9, 3.1, 1.5, 0.1, which makes them look like a duplicate pair. They are not duplicates. They are two distinct flowers, one of which was copied wrongly. The corrected values are also distributed by UCI as `bezdekIris.data`, which allowed the fix to be verified against a second source rather than trusted. The correction function checks that each target row still holds the known-incorrect values before overwriting it, so it fails loudly instead of silently corrupting good data if it is ever pointed at a different file.

The third problem was the one duplicate that survives the correction: sample 143 is an exact copy of sample 102. Both are real virginica flowers that happen to share all four measurements at the recorded precision of 0.1 cm. This row was deliberately kept. The audit function reports duplicates but never removes them, because whether a repeated row is a recording error or a coincidence is a question about the world, not a question that can be settled from the table. Dropping it would have left virginica with 49 flowers against 50 for each of the other species,

introducing an imbalance that the source data does not have.

Exploratory analysis: separation, not just difference

A table of group means can show that two species differ on average while saying nothing about whether their distributions actually come apart. Because the project's question is about distinguishing species, the analysis therefore reports a standardised mean difference: for each measurement and each pair of species, the absolute difference in means divided by the pooled standard deviation. The result is unitless, so petal width and sepal length can be compared directly on the same scale, and it answers how cleanly two species separate rather than merely whether their averages differ. The assumption behind pooling is that the two groups have broadly comparable spread; setosa's petal measurements are noticeably less variable than the other species', so these values are treated as indicative rather than exact.

Visualization: one question per figure

Each figure was designed to answer a single question, and the figures were built on a shared visual system defined once at the top of the notebook so that no individual chart invents its own conventions.

- Figure 1 asks whether petal measurements separate the species better than sepal measurements. It places the two measurement spaces side by side so the shapes of the point clouds can be compared directly.
- Figure 2 asks how the four measurements differ across species including their spread. It uses small multiples rather than one panel, because the four measurements have different ranges and a shared axis would compress the petal widths into a sliver. Individual flowers are drawn behind each box so that sample size and distribution shape stay visible, which a box alone conceals.
- Figure 3 asks which measurements carry overlapping information. The colour scale is diverging, with one hue for positive correlation, another for negative, and a neutral grey at exactly zero. It is fixed to the full range from -1 to +1 so that the neutral midpoint falls on zero rather than on the middle of whatever happens to be in this dataset, which would have made weak correlations look neutral.
- Figure 4 asks whether the separation is a genuine gap or merely a shift in average, a distinction that decides whether one measurement could classify a flower on its own.

Two accessibility decisions apply throughout. Species is encoded by marker shape as well as colour, so the figures remain readable in greyscale, in print, and to a reader with colour-vision deficiency; colour is never the only carrier of meaning. The three species colours were checked computationally for colour-vision-deficiency separation rather than chosen by eye. In addition, every pattern visible in a figure has an exact numerical counterpart in the tables produced in the analysis section, so no value is reachable only by reading a picture.

Results and Interpretation

The four measurements do not contribute equally to telling the species apart, and the gap between them is large. Measured as standardised mean differences, setosa differs from virginica by about ten pooled standard deviations in petal length but by only about three in sepal length; the corresponding figure for sepal width is 1.3. A separation of ten pooled standard deviations is not a tendency, it is a clean split.

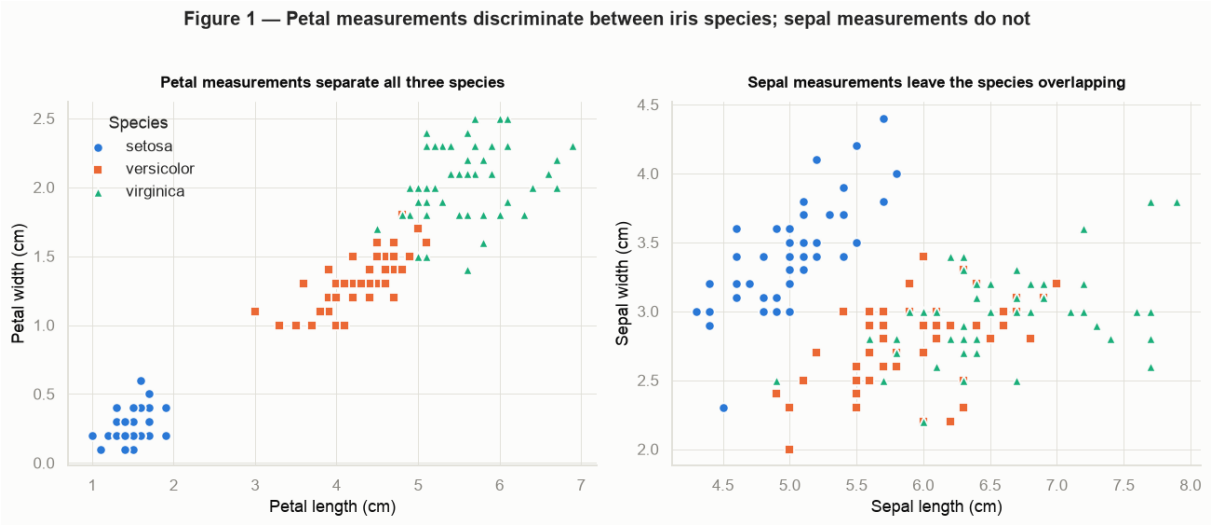


Figure 1. Petal measurements separate all three species; sepal measurements leave them overlapping.

Figure 1 makes the asymmetry visible. In petal space the three species form three distinguishable groups; in sepal space they form a single cloud in which versicolor and virginica are thoroughly mixed. The figure also corrects a common oversimplification. The species are not symmetrically arranged: setosa is cleanly separated from both others, while versicolor and virginica overlap each other in every one of the four measurements. The dataset really contains one easy discrimination problem and one hard one.

Figure 2 adds the within-group spread and reveals a pattern that the petal measurements alone would hide. Sepal width is the only measurement on which setosa scores highest rather than lowest. Setosa is therefore not simply a smaller flower; it has short, narrow petals on relatively wide sepals, which is a genuinely different shape rather than a scaled-down version of the same one.

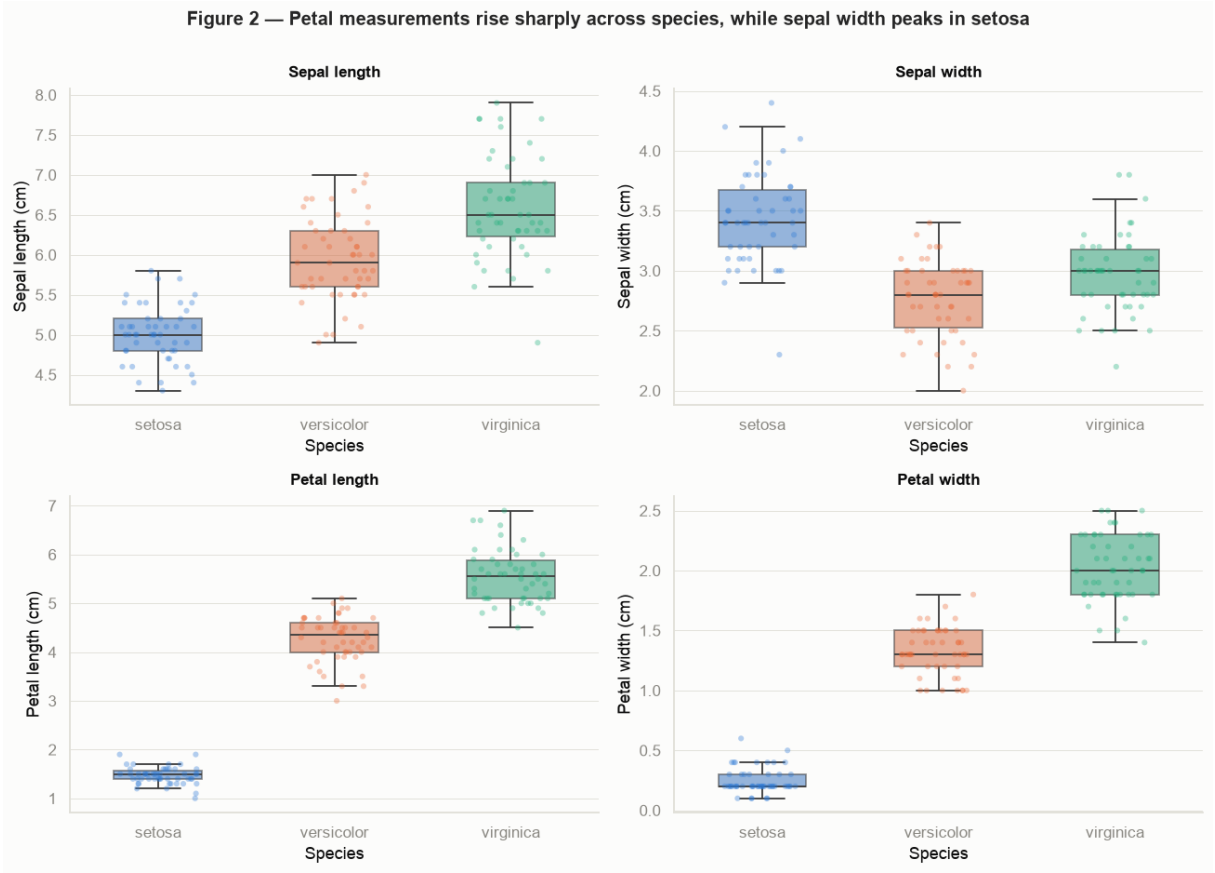


Figure 2. Distribution of each measurement by species. Individual flowers are drawn behind each box so that sample size and distribution shape remain visible.

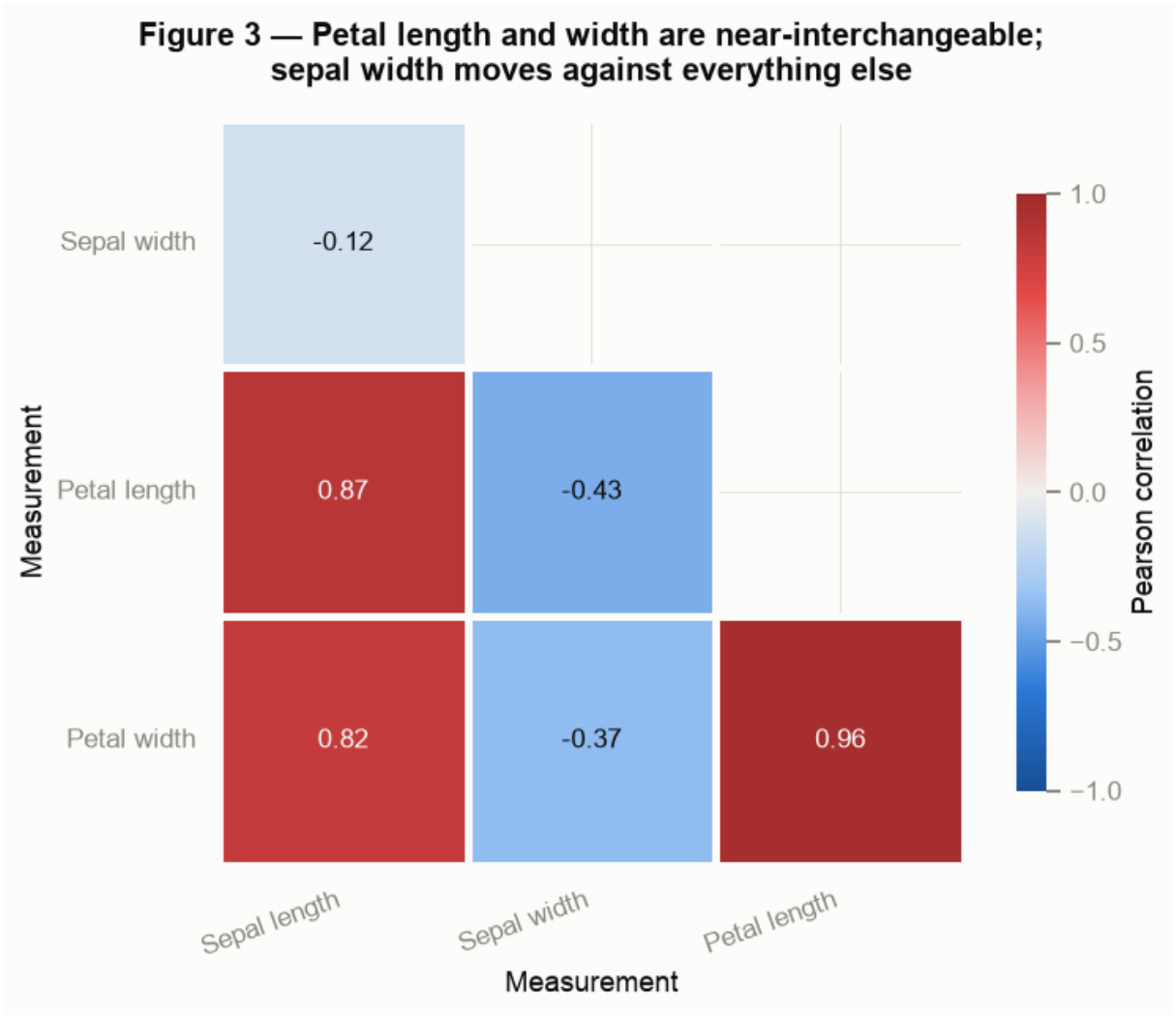


Figure 3. Petal length and petal width are near-interchangeable (r = 0.96); sepal width moves against the other three measurements.

Figure 3 explains why four measurements do not supply four independent pieces of evidence. Petal length and petal width correlate at 0.96, high enough that they are nearly interchangeable; the apparent two-dimensionality of Figure 1's left panel is largely redundant, since the points fall close to a line. Sepal width is the only measurement that is negatively correlated with the others, which is the same shape finding seen in Figure 2 expressed as a correlation.

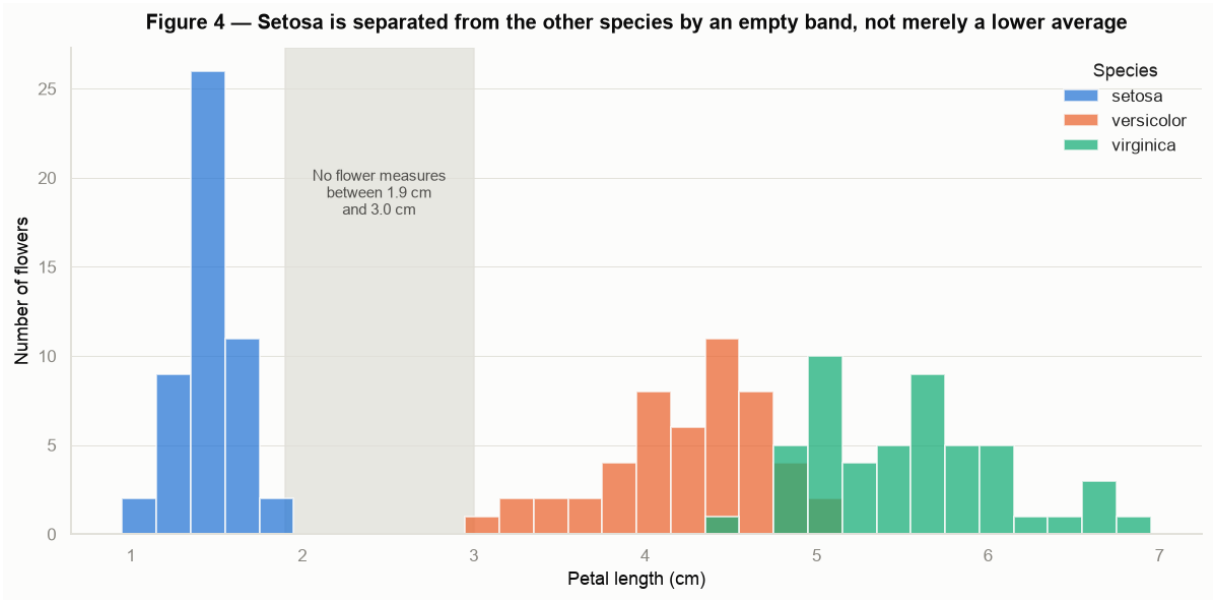


Figure 4. No flower in the dataset has a petal length between 1.9 cm and 3.0 cm, so setosa is separated from the other two species by an empty band rather than by a difference in averages.

Figure 4 sharpens the central result. Setosa's petals never exceed 1.9 cm in length, and no versicolor or virginica flower measures below 3.0 cm. The 1.1 cm band between those values is completely unoccupied. This is a stronger claim than saying setosa has shorter petals on average: a single measurement would classify every setosa in this sample correctly, and a plot of group averages alone would have concealed the distinction.

Responsible Practice: Bias and Data Quality

The clearest place this analysis could have gone wrong is the duplicate handling, and it is worth being precise about why. A routine cleaning pipeline that calls a `drop_duplicates` step as a matter of habit would have removed three rows from this dataset. Two of those rows were not duplicates at all; they were distinct flowers rendered identical by a documented transcription error, and deleting one of them would have destroyed a real observation while appearing to improve data quality. The third is a real repeated measurement whose status cannot be determined from the data. Reading the documentation distributed with the dataset was what distinguished these cases, and the general lesson is that the documentation shipped with data is part of the data.

Silent, automated deletion is the specific risk. Because the two corrupted rows both belonged to setosa, dropping them would have shrunk one class relative to the others and biased any subsequent per-species comparison, all without producing a single error or warning. The mitigation adopted here is to separate detection from action: the audit function reports missing values, physically impossible measurements, and duplicates, but it removes nothing. Every change to the data is made by an explicitly named function, is verified against the known prior state before it is applied, prints what it changed, and is justified in prose in the notebook.

A second and more fundamental risk concerns what this sample can support. The data is small, curated, and old: exactly 50 flowers per species, collected in the mid-1930s, with two of the three species gathered from the same pasture on the same day. The perfectly equal class sizes are a property of the study design, not of nature, and the clean separation found here should not be read as a general fact about irises in the wild. Reporting a classification accuracy from this dataset without that caveat would overstate what the evidence shows. Nothing in the notebook

extrapolates beyond the sample.

Two narrower assumptions are also recorded. Pearson correlation measures linear association only, so Figure 3 would understate a relationship that is strong but curved. And the recording precision of 0.1 cm is coarse relative to the differences between versicolor and virginica, which is the direct cause of the surviving duplicate: two distinguishable flowers can easily round to the same four values. As a next step, the sensitivity of the conclusions to the retained duplicate could be checked by re-running the analysis with it excluded and confirming that nothing material changes.

Reproducibility

Reproducing this analysis requires four steps: clone the repository, create a virtual environment, install the pinned dependencies with `pip install -r requirements.txt`, and run the notebook from top to bottom. The README gives the exact commands. The notebook requires no network access and no manual intervention; it reads only `iris.csv` from the project folder, and the dataset is committed to the repository rather than downloaded at runtime, so the input cannot change underneath the analysis.

Reading a committed file rather than fetching one is a deliberate trade-off, and it has a cost worth naming: the local copy could silently fall behind the published dataset. The notebook therefore includes a `refresh_from_source` function that re-downloads the published file, compares it against a recorded SHA-256 digest, and rewrites the local CSV only if the two match. It is deliberately never called during the analysis, so the run stays offline and deterministic; it exists so that the committed copy can be audited on demand rather than trusted indefinitely. Recording the digest rather than merely computing one is what makes the check meaningful, since a revision upstream then becomes a reviewable edit to the notebook instead of an invisible change in the results.

The dependency file, `requirements.txt`, pins an exact version of each package the notebook uses (NumPy, pandas, Matplotlib, seaborn, and the Jupyter kernel) rather than accepting whatever version happens to be current. This matters more than it may appear. Danchev (2022) notes that a platform supplying its own pre-installed library versions, instead of rebuilding from a pinned environment specification, introduces possible errors; recording dependencies explicitly with a package manager is also the sixth of the ten rules in Rule et al. (2019). The notebook reinforces this by printing the versions of NumPy, pandas, Matplotlib, and seaborn as its first output, so every rendered copy carries a record of the environment that produced it.

The work is version-controlled in Git. Development took place on a `dev` branch separate from `main`, with commits following the stages of the workflow, so the history shows the analysis being built rather than appearing complete in a single commit. Work reached `main` through pull requests, so each change could be reviewed before being merged. Rule et al. (2019) recommend precisely this, version-controlling notebooks and sharing them in a repository that contains the data and the environment specification alongside the code.

One honest caveat belongs here. A notebook is not reproducible merely because it is a notebook; cells can be executed out of order, leaving state that no fresh run would produce. Danchev (2022), citing Rule et al. (2019), makes this point directly: notebooks foster good practice but do not guarantee reproducibility. The guarantee has to come from discipline. Here that discipline is a notebook whose every cell depends only on cells above it, and which was verified by executing it end to end from a clean kernel; the outputs committed to the repository are the outputs of such a run.

References

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education*, 5(56), 156. https://doi.org/10.21105/jose.00156

Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188. https://doi.org/10.1111/j.1469-1809.1936.tb02137.x

Git. (n.d.). *git-branch documentation*. Retrieved 17 October 2026, from https://git-scm.com/docs/git-branch

Hunter, J. D., and the Matplotlib development team. (n.d.). *Matplotlib documentation*. Retrieved 17 October 2026, from https://matplotlib.org/stable/

Rule, A., Birmingham, A., Zuniga, C., Altintas, I., Huang, S.-C., Knight, R., Moshiri, N., Nguyen, M. H., Rosenthal, S. B., Perez, F., and Rose, P. W. (2019). Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks. *PLOS Computational Biology*, 15(7), e1007007. https://doi.org/10.1371/journal.pcbi.1007007

The pandas development team. (n.d.). *pandas documentation*. Retrieved 17 October 2026, from https://pandas.pydata.org/docs/

UCI Machine Learning Repository. (n.d.). *Iris* [Data set]. Retrieved 17 October 2026, from https://archive.ics.uci.edu/dataset/53/iris